Universidade Federal de Minas Gerais
Escola de Engenharia
EEE048 Fundamentos de Inteligência Artificial

Relatório do Trabalho Prático 2 (TP2): Busca Local e Algoritmo Genético

Cecília Rodrigues Saraiva
Caio César Araújo Guimarães
Stéphanie Pereira Barbosa
Thales Eduardo Dias de Souza

23 de outubro de 2026

Sumário

Introdução **3**

Modelagem **3**

Algoritmos **4**
0.1 (A) *Greedy Descent* com *Random Restart* 4
0.2 (B) *Simulated Annealing* 4
0.3 (C) *Genetic Algorithm* 4

Protocolo Experimental **5**

Análise Comparativa **7**
0.4 Ótimos Locais 10
0.5 Exploração x intensificação 12
0.6 Efeito da Escala 14
0.7 Efetividade do *Genetic Algorithm* 14

Conclusões **15**

CRediT – Taxonomia de Contribuição de Autor **15**

Declaração de Uso de IA Generativa **15**

Referências **16**

Introdução

Esse trabalho tem como objetivo utilizar um problema de engenharia real como meio de comparação de heurísticas de busca local e uma meta-heurística populacional que é o Algoritmo Genético. A ideia é utilizar três tamanhos amostrais de forma tal que possamos observar, na prática, os comportamentos dos paradigmas de exploração e intensificação.

O problema abordado é o sequenciamento de manutenção de ativos de uma dada concessionária de distribuição de energia em que temos n ativos, sendo eles religadores, transformadores de distribuição, entre outros dispostos em uma determinada área. Consideramos somente uma equipe responsável pela manutenção dos ativos em que a equipe trabalha em um ativo por vez. A caracterização dos j ativos pode ser dada da seguinte maneira:

- a **posição** (x_j, y_j)
- **duração do serviço** p_j (em horas)
- o **prazo regulatório** d_j (hora-limite para a conclusão)
- **peso de criticidade** w_j (custo por hora de atraso)

Temos também o deslocamento entre dois pontos i e j dado por:

$$t_{ij} = \text{round}\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right) \text{ horas}, \tag{1}$$

com arredondamento para o inteiro mais próximo. A base é o ponto de índice 0, com $p_0 = 0$. A equipe não precisa retornar à base.

Modelagem

Em relação à modelagem do problema, temos que uma possível solução é uma permutação $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ dos n ativos: a ordem de visita. A conclusão do k-ésimo ativo visitado é dada pela equação:

$$C_{\pi_k} = C_{\pi_{k-1}} + t_{\pi_{k-1}\pi_k} + p_{\pi_k}, \tag{2}$$

com $\pi_0 = 0$ e $C_{\pi_0} = 0$.

Assim, temos que a **função de avaliação** *(fitness)* a ser **minimizada** é o atraso ponderado total:

$$h(\pi) = \sum_{j=1}^{n} w_j \cdot \max(0, C_j - d_j). \tag{3}$$

Algumas observações úteis dadas pelo enunciado são:

- É um problema de otimização combinatória com espaço $n!$.
- Não é um CSP (*Constraint Satisfaction Problem*), uma vez que todas as permutações são viáveis, ou seja, não há *hard constraints*. Dessa forma, podemos afirmar que o problema é somente de *soft constraints*, já que estamos considerando apenas a minimização da função de custo.
- h não decresce durante a permutação, ou seja, os custos dos primeiros k ativos não diminuem para o $(k+1)$-ésimo.

Algoritmos

Nesse trabalho estaremos utilizando e comparando três algoritmos vistos em sala de aula, sendo eles o *Greedy Descent* com *Random Restart*, o *Simulated Annealing* e o *Genetic Algorithm*, para resolver esse problema de otimização combinatória. O objetivo, como já mencionado anteriormente, é comparar os paradigmas de exploração e intensificação para um problema real e para esse problema adotaremos a seguinte restrição. Os três algoritmos **não podem conhecer o problema** que estão resolvendo *a priori*, ou seja, apenas irão interagir com ele por meio de uma interface comum.

0.1 (A) *Greedy Descent* com *Random Restart*

O algoritmo *Greedy Descent* com *Random Restart* tem como característica buscar a melhor solução explorando repetidamente a vizinhança de um ponto inicial gerado aleatoriamente e movendo-se sempre para o vizinho de menor custo (descida gulosa) até atingir um mínimo local, de tal forma que o processo é reiniciado a partir de uma nova semente para escapar de ótimos locais.

Nesse problema ele está **sujeito a**:
implementar e comparar pelo menos **duas das três** variantes de seleção do próximo sucessor, ou seja:

- ***Best improvement***: varre toda a vizinhança e escolhe o melhor sucessor;
- ***First improvement***: aceita o primeiro sucessor que melhora h;
- **Sucessor aleatório com aceitação de melhora**: sorteia um único sucessor e o aceita apenas se melhorar h.

0.2 (B) *Simulated Annealing*

O *Simulated Annealing* é uma meta-heurística baseada no recozimento metalúrgico que aceita melhorias de forma gulosa, mas também permite, com uma probabilidade controlada por um parâmetro de temperatura decrescente, a aceitação de soluções piores, o que viabiliza a transposição de barreiras e o escape de mínimos locais.

Que está **sujeito a**:
Seja π a solução corrente e π' um sucessor sorteado ao acaso:

- se $h(\pi') \leq h(\pi)$, aceite π';
- caso contrário, aceite π' com probabilidade $e^{-(h(\pi')-h(\pi))/T}$ (distribuição de Gibbs).

0.3 (C) *Genetic Algorithm*

O *Genetic Algorithm* (Algoritmo Genético) se baseia na simulação do processo de evolução natural utilizando uma população de soluções candidatas que são iterativamente combinadas (*crossover*), mutadas e selecionadas por meio de uma função de aptidão (*fitness*), promovendo a exploração e a explotação do espaço de busca ao longo de gerações.

Que está **sujeito a**:
Uma população de tamanho N, operadores de seleção, cruzamento com probabilidade P_c, mutação com probabilidade P_m e elitismo, seguindo o ciclo de vida e o pseudocódigo vistos em aula.

- **Inicialização**: amostragem aleatória de N sequenciamentos.
- **Seleção**: por torneio ou proporcional à aptidão (roleta).
- **Cruzamento**: use um operador que preserve a validade da permutação — o *Order Crossover* (OX) visto em aula, o *Partially Mapped Crossover* (PMX) ou o cruzamento por arestas.
- **Mutação**: aplique uma pequena taxa de mutação para inserir diversidade e favorecer a exploração. A *swap mutation* é o ponto de partida inicial.
- **Critério de parada**: neste trabalho, o critério é o esgotamento do orçamento de avaliações (Seção 3), de modo que a comparação entre os três algoritmos seja justa.

Protocolo Experimental

Instâncias e orçamento

Foram utilizadas três instâncias do problema, todas com a base localizada em $(50, 50)$. O critério de parada comum aos três algoritmos é o esgotamento de um orçamento de avaliações da função h, de modo que a comparação seja justa entre métodos que gastam o orçamento de maneiras diferentes. A Tabela 1 resume as instâncias, os orçamentos e o número de repetições independentes.

Tabela 1: Instâncias, orçamento de avaliações e repetições por configuração.

Instância	Ativos (n)	Orçamento de avaliações	Repetições
Pequena	12	50.000	30
Média	50	150.000	30
Grande	150	300.000	10

O orçamento contabiliza todas as chamadas à função h, incluindo as avaliações das soluções iniciais e, no *Simulated Annealing*, as avaliações usadas na calibração da temperatura. O código verifica, a cada execução, que o número de avaliações realizadas é exatamente igual ao orçamento previsto e que o custo retornado coincide com o recalculado a partir da melhor solução. Cada execução utiliza a semente $20261013 + r$, em que r é o índice da repetição (de 0 a 29, ou de 0 a 9 na instância grande), e as sementes ficam registradas no arquivo `execucoes.csv`, o que permite reproduzir os resultados.

Configurações avaliadas

Foram avaliadas seis configurações por instância, totalizando $6 \times (30+30+10) = 420$ execuções:

1. **GD**: *Greedy Descent* com *Random Restart*, *best improvement* e vizinhança *insertion*;
2. **GD_first**: *first improvement* com vizinhança *insertion*;
3. **GD_swap**: *best improvement* com vizinhança *swap*;
4. **GD_2opt**: *best improvement* com vizinhança 2-opt;
5. **SA**: *Simulated Annealing* com vizinhança *insertion*;

6. **GA**: Algoritmo Genético com *Order Crossover* (OX).

Dessa forma, das três variantes de seleção de sucessor do *Greedy Descent*, foram comparadas *best improvement* e *first improvement*. Embora o código também implemente o sucessor aleatório com aceitação de melhora, o PMX e a seleção por roleta, essas opções não fazem parte do protocolo oficial e não há resultados delas neste relatório.

Vizinhanças

Para uma permutação de n ativos, as vizinhanças utilizadas são:

- ***Swap***: troca a posição de dois ativos, com $n(n-1)/2$ vizinhos;
- ***Insertion***: remove um ativo da posição i e o reinsere na posição $j \neq i$, com $n(n-1)$ vizinhos;
- **2-opt**: inverte o trecho da sequência entre as posições i e j, com $n(n-1)/2$ vizinhos.

Parâmetros dos algoritmos

- ***Greedy Descent***: a cada reinício a solução inicial é uma permutação aleatória; a descida termina ao atingir um ótimo local (nenhum vizinho melhora h) e, então, um novo reinício é feito enquanto houver orçamento.
- ***Simulated Annealing***: sucessor sorteado ao acaso na vizinhança *insertion*. A temperatura inicial T_0 é calibrada antes da busca por uma caminhada aleatória de até 200 passos: toma-se a mediana dos aumentos de custo observados e define-se T_0 de modo que um aumento típico seja aceito com probabilidade de aproximadamente 95%, com $T_0 \geq 1$. O resfriamento é geométrico, com fator calculado para que a temperatura vá de T_0 até $T_f = \max(0{,}01;\ 10^{-4}T_0)$ ao longo de todo o orçamento restante. Os valores de T_0 calibrados variaram de cerca de 8.832 a 13.706 (pequena), 41.760 a 72.856 (média) e 102.918 a 167.585 (grande).
- **Algoritmo Genético**: população de $N = 50$ indivíduos, seleção por torneio com $k = 3$, cruzamento OX com $P_c = 0{,}8$, *swap mutation* com $P_m = 0{,}1$ e elitismo de 2 indivíduos. Os elites são copiados com seus custos já conhecidos, sem nova avaliação.

Avaliação incremental

Nos movimentos do *Greedy Descent* foi implementada a avaliação incremental de h, que reaproveita o prefixo da sequência que não foi alterado e recalcula apenas o sufixo a partir da primeira posição diferente. Os testes automatizados conferem que ela fornece o mesmo valor da avaliação completa, e cada avaliação incremental também é contabilizada no orçamento. Um *benchmark* com 2.000 vizinhos por caso (Tabela 2) mostrou que o ganho de tempo variou conforme a instância e a vizinhança: a avaliação incremental foi mais lenta na instância pequena, mais rápida na média e teve resultado misto na grande.

Tabela 2: Tempo (s) das avaliações completa e incremental para 2.000 vizinhos e razão incremental/completa.

Instância	Vizinhança	Completa	Incremental	Razão
Pequena	*swap*	0,0141	0,0170	1,21
Pequena	*insertion*	0,0139	0,0185	1,33
Pequena	2-opt	0,0176	0,0216	1,22
Média	*swap*	0,0740	0,0619	0,84
Média	*insertion*	0,0619	0,0572	0,92
Média	2-opt	0,0803	0,0673	0,84
Grande	*swap*	0,1272	0,1324	1,04
Grande	*insertion*	0,1564	0,1357	0,87
Grande	2-opt	0,1378	0,2260	1,64

Métricas e testes estatísticos

A qualidade de cada execução é o menor valor de h encontrado (quanto menor, melhor). Para cada configuração e instância são reportados média, desvio padrão amostral, mínimo, mediana e tempo médio de execução. Na instância pequena também é reportado o *gap* médio em relação a $h = 1.875$, o melhor valor encontrado por todos os algoritmos e usado como referência no código; este trabalho não prova que esse valor seja o ótimo global. Para comparar GD, SA e GA, aplicou-se o teste bilateral de Mann–Whitney, e os p-valores são apresentados sem correção para múltiplas comparações. Como referência de validação, a permutação identidade na instância pequena tem $h = 4.366$, valor conferido pelos testes automatizados.

Análise Comparativa

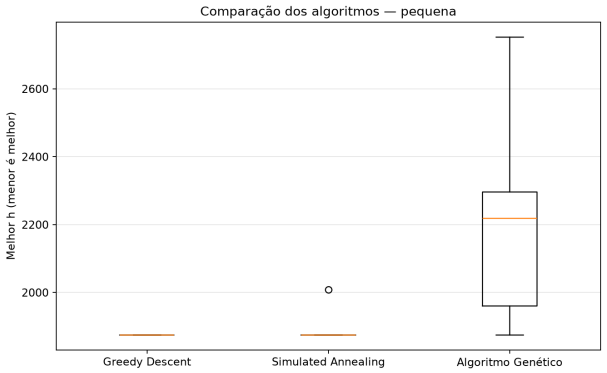
As Tabelas 3 e 4 resumem os resultados das 420 execuções. As Figuras 1 e 2 mostram a distribuição do melhor h e a convergência mediana dos três algoritmos principais.

Tabela 3: Melhor h obtido por configuração (30 repetições nas instâncias pequena e média, 10 na grande).

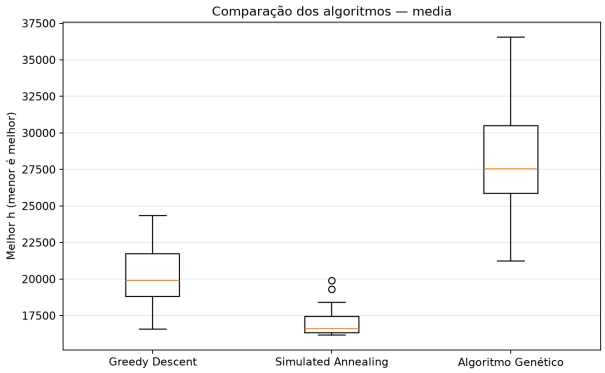
Instância	Configuração	Média	Desvio	Mínimo	Mediana	Tempo médio (s)
Pequena	GD	1.875,0	0,0	1.875	1.875	0,53
	SA	1.879,5	24,5	1.875	1.875	0,61
	GA	2.186,9	235,9	1.875	2.219	1,25
	GD_swap	1.875,0	0,0	1.875	1.875	0,41
	GD_2opt	1.875,0	0,0	1.875	1.875	0,43
	GD_first	1.879,1	15,7	1.875	1.875	0,40
Média	GD	20.207,2	2.220,7	16.571	19.892,5	3,94
	SA	17.017,2	982,4	16.175	16.594,5	3,79
	GA	28.223,5	3.229,0	21.233	27.557	11,08
	GD_swap	29.197,0	3.356,8	21.638	29.111	4,65
	GD_2opt	20.105,1	1.278,4	17.250	20.128	4,27
	GD_first	20.018,0	1.727,1	17.152	19.696	7,42
Grande	GD	777.504,0	45.068,0	709.286	787.449,5	32,45
	SA	70.817,6	3.529,4	66.947	70.540,5	20,78
	GA	162.598,7	12.780,2	149.335	161.694	161,76
	GD_swap	473.735,8	13.001,9	454.526	476.103	27,92
	GD_2opt	479.026,9	16.308,7	443.725	482.642	29,33
	GD_first	678.745,4	45.379,3	581.467	690.030,5	28,15

Tabela 4: P-valores do teste bilateral de Mann–Whitney (sem correção para múltiplas comparações).

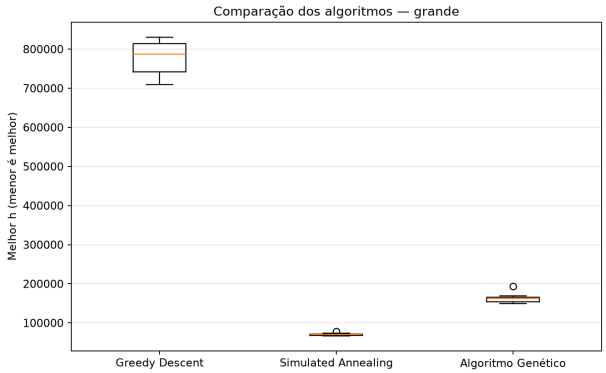
Instância	GD × SA	GD × GA	SA × GA
Pequena	0,334	$1{,}9 \times 10^{-10}$	$5{,}3 \times 10^{-10}$
Média	$1{,}5 \times 10^{-8}$	$9{,}0 \times 10^{-11}$	$3{,}0 \times 10^{-11}$
Grande	$1{,}8 \times 10^{-4}$	$1{,}8 \times 10^{-4}$	$1{,}8 \times 10^{-4}$



(a) Pequena

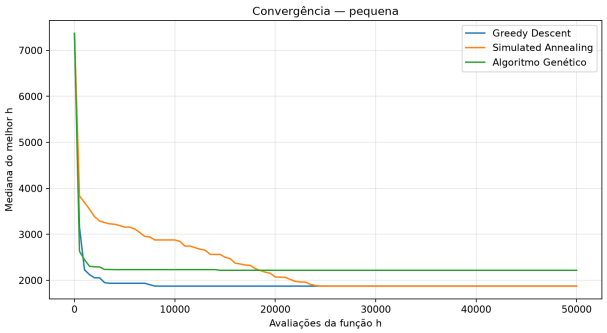


(b) Média

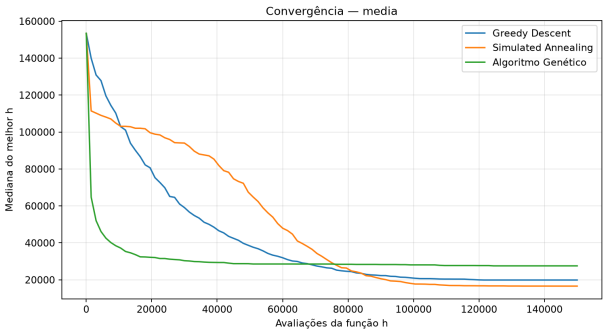


(c) Grande

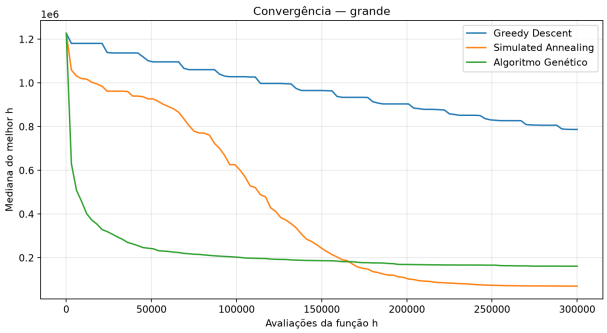
Figura 1: Distribuição do melhor h obtido por GD, SA e GA em cada instância.



(a) Pequena



(b) Média



(c) Grande

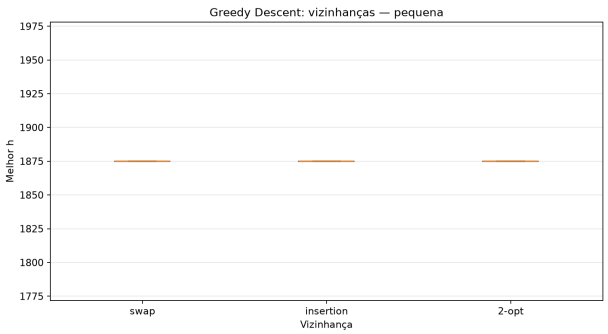
Figura 2: Mediana, entre as repetições, do melhor h em função do número de avaliações.

0.4 Ótimos Locais

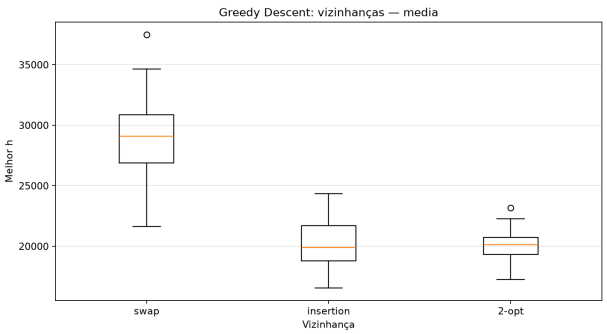
Na instância pequena, o *Greedy Descent* com *best improvement* e *insertion* atingiu $h = 1.875$ nas 30 repetições (desvio nulo), assim como as variantes *swap* e 2-opt. A variante *first improvement* atingiu esse valor na mediana, mas com média ligeiramente acima (1.879,1). Em média, o GD realizou cerca de 45,6 reinícios por execução e encontrou cerca de 12,7 ótimos locais distintos, o que indica que muitas descidas terminam em ótimos locais já visitados. Com a vizinhança *swap*, foram cerca de 89,4 reinícios e 60,7 ótimos locais distintos; com 2-opt, 82,3 reinícios e 27,9 distintos; e com *first improvement*, 47,7 reinícios e 13,2 distintos. Ou seja, a quantidade de ótimos locais distintos encontrados depende da vizinhança e da estratégia de seleção.

À medida que o problema cresce, o comportamento muda. Na instância média, o GD com *insertion* fez em média apenas 2,0 reinícios e atingiu cerca de 1,0 ótimo local por execução; nas variantes *swap* e 2-opt foram 3,0 reinícios e 2,0 ótimos locais. Na instância grande, o GD ficou em média em 1,0 reinício e **nenhuma** descida chegou a um ótimo local dentro do orçamento (média de 0,0 ótimos locais em todas as variantes). Isso é coerente com o custo de varrer a vizinhança: com *best improvement*, cada passo exige $n(n-1)$ avaliações na vizinhança *insertion* (22.350 para $n = 150$), de modo que 300.000 avaliações permitem apenas cerca de 13 passos, valor que coincide com a média de 13,0 passos de melhoria registrada. Na vizinhança *swap* ou 2-opt, com $n(n-1)/2 = 11.175$ vizinhos, foram 26 passos. Portanto, na instância grande o desempenho do GD é limitado pelo orçamento, e não pela presença de ótimos locais.

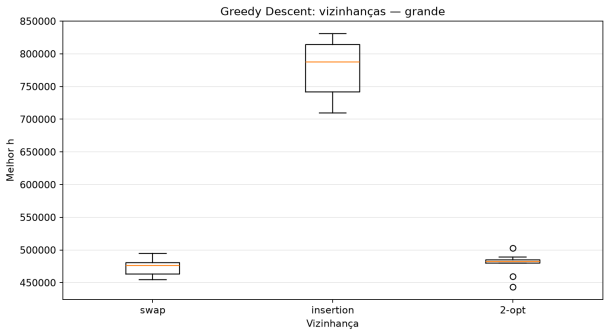
Os gráficos de comparação entre as vizinhanças e entre as estratégias do GD estão nas Figuras 3 e 4. Na instância média, a vizinhança *swap* foi claramente pior (média de 29.197,0) que *insertion* (20.207,2) e 2-opt (20.105,1), mesmo realizando mais passos (média de 120 contra 60). Na instância grande, *swap* e 2-opt (médias de 473.735,8 e 479.026,9) ficaram melhores que *insertion* (777.504,0), e *first improvement* (678.745,4) também superou *best improvement* com *insertion*, ambos com 10 repetições. Os dados disponíveis não permitem afirmar a causa dessas diferenças além da observação de que, na instância grande, todas as descidas foram interrompidas pelo orçamento antes de atingir um ótimo local. Não foi feito teste estatístico entre as variantes do GD.



(a) Pequena

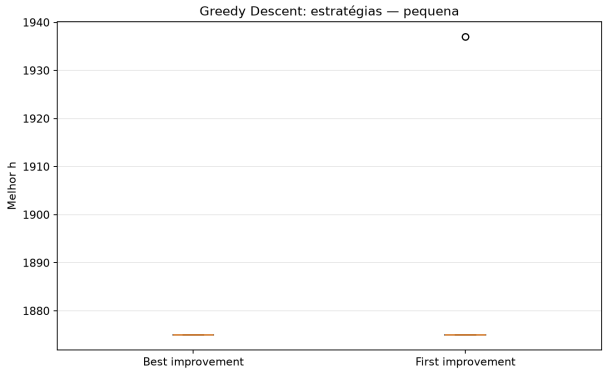


(b) Média

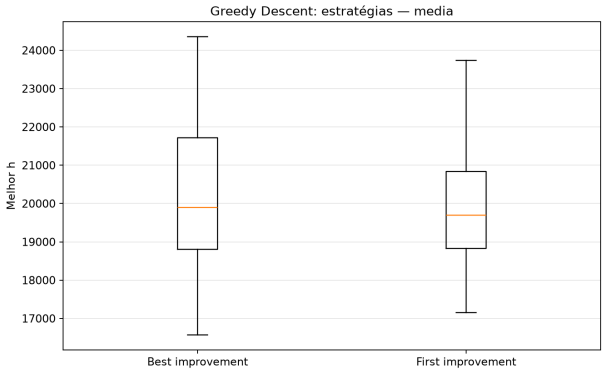


(c) Grande

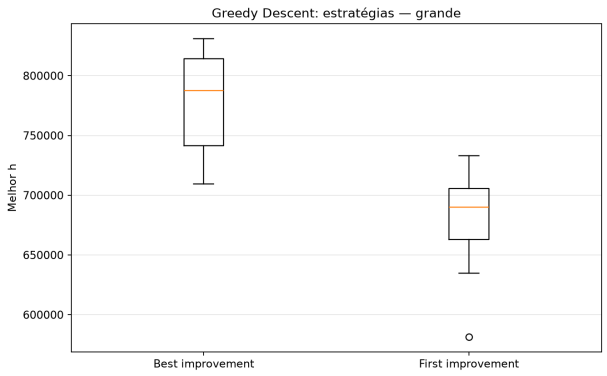
Figura 3: *Greedy Descent*: comparação das vizinhanças *swap*, *insertion* e 2-opt (*best improvement*).



(a) Pequena



(b) Média



(c) Grande

Figura 4: *Greedy Descent*: *best improvement* contra *first improvement* (vizinhança *insertion*).

0.5 Exploração x intensificação

O *Greedy Descent* é essencialmente intensificação: só aceita melhorias e a diversificação vem exclusivamente dos reinícios aleatórios. Como visto acima, nas instâncias média e grande o número de reinícios é muito pequeno (2,0 e 1,0 em média), de modo que praticamente não há exploração.

No *Simulated Annealing*, o controle entre os dois paradigmas é feito pela temperatura. A taxa de aceitação de movimentos piores, registrada ao longo da execução (Figura 5), começa em torno de 0,93–0,94 nas três instâncias (comportamento de exploração, próximo de uma caminhada aleatória), cai para cerca de 0,05–0,08 na metade do orçamento e chega a aproximadamente 0,001 ao final nas instâncias média e grande (0,0 na pequena), quando a busca passa a se comportar como uma descida gulosa. Nas curvas de convergência (Figura 2) isso aparece na instância grande como uma queda lenta no início e mais acentuada na fase intermediária, em que o SA ultrapassa o GA por volta de 170.000 avaliações.

No Algoritmo Genético, o indicador utilizado é a diversidade, definida como a fração de indivíduos distintos na população. Ela começa em 1,0 (população inicial aleatória) e cai rapidamente, ficando em torno de 0,12–0,13 durante o restante da execução nas três instâncias (Figura 5). Além disso, na instância média a média da população ao final (28.913,2) é muito próxima da média dos melhores valores obtidos (28.223,5), e na instância grande são 164.706,3 e 162.598,7, respectivamente, o que indica que a população se concentrou em torno de soluções de qualidade parecida (Figura 6). Isso sugere que a intensificação do GA domina a maior parte da execução, com a exploração restrita ao início e à mutação com $P_m = 0{,}1$.

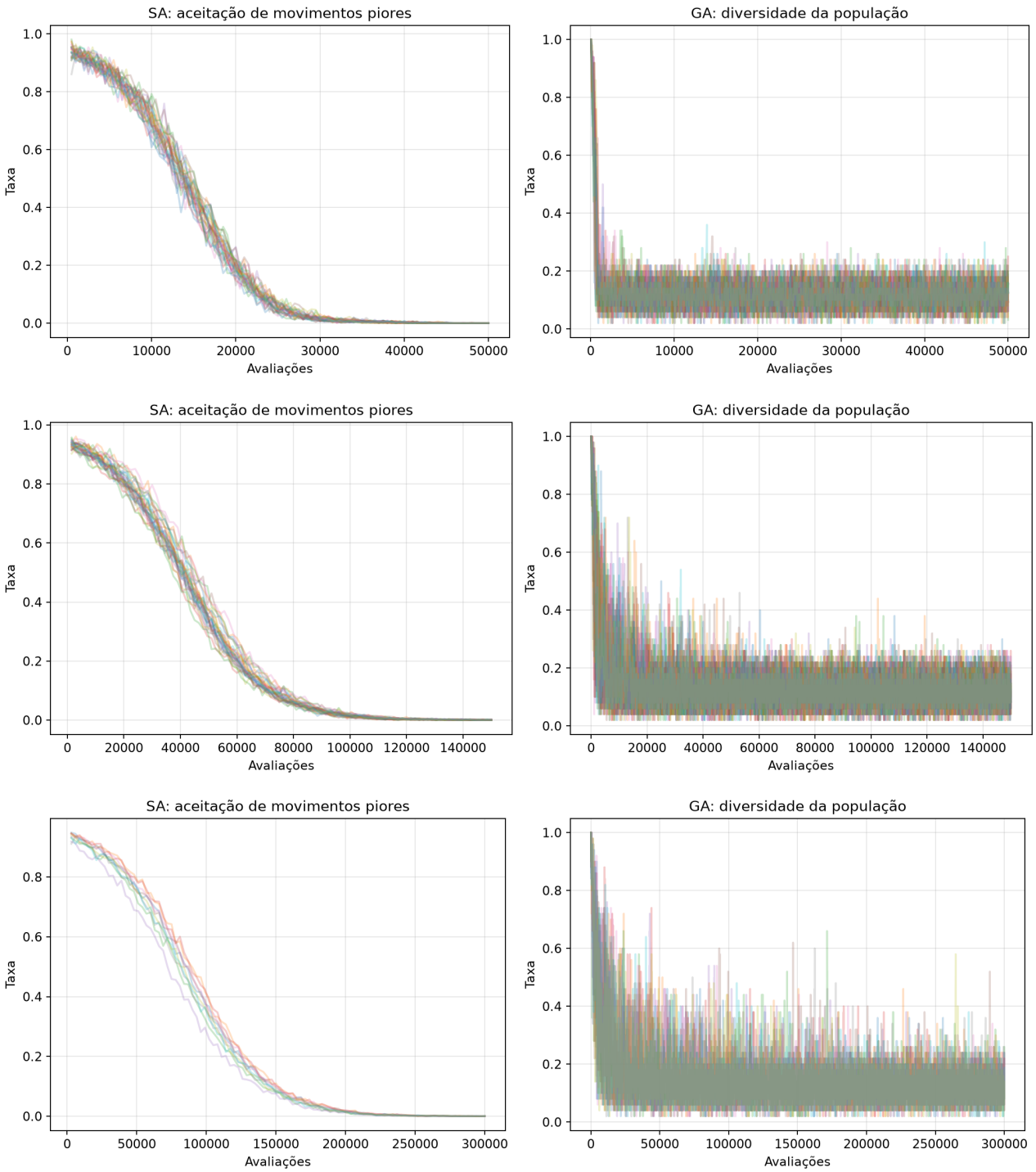
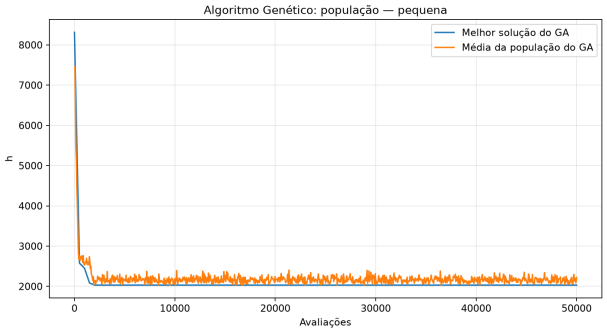
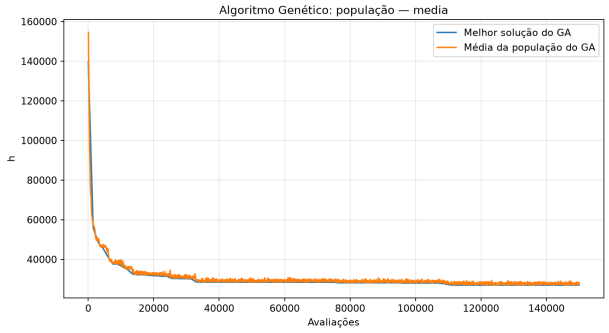


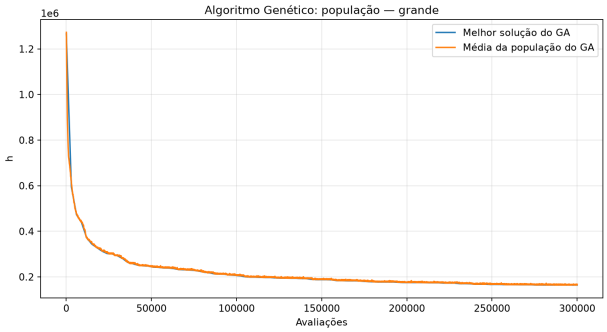
Figura 5: Taxa de aceitação de movimentos piores no SA (esquerda) e diversidade da população no GA (direita), nas instâncias pequena, média e grande, com uma curva por repetição.



(a) Pequena



(b) Média



(c) Grande

Figura 6: Algoritmo Genético: melhor solução e média da população (primeira repetição).

0.6 Efeito da Escala

Na instância pequena ($12! \approx 4{,}8 \times 10^8$ permutações), GD e SA são estatisticamente indistinguíveis ($p = 0{,}334$), ambos chegando a 1.875 em todas ou quase todas as repetições (média do SA de 1.879,5, com desvio de 24,5), enquanto o GA tem média de 2.186,9, cerca de 16,6% acima da referência. Na instância média, o SA foi o melhor (média de 17.017,2), seguido pelo GD (20.207,2) e pelo GA (28.223,5), com diferenças significativas entre todos os pares (Tabela 4). Na instância grande, a ordem muda: SA (70.817,6), GA (162.598,7) e GD (777.504,0), novamente com $p = 1{,}8 \times 10^{-4}$ em todos os pares.

Em termos relativos ao SA, a média do GD é 1,19 vez a do SA na instância média e cerca de 11,0 vezes na grande, enquanto a do GA é 1,66 e cerca de 2,3 vezes, respectivamente. Ou seja, a diferença entre o SA e os demais cresce com o tamanho da instância. Para o GD, isso se relaciona com o que foi mostrado na seção de ótimos locais: o custo de varrer a vizinhança cresce com n e o orçamento deixa de permitir a convergência. O SA, por usar um único sucessor sorteado por avaliação, provavelmente aproveita melhor o orçamento nessas escalas. O GA piorou em relação ao SA com a escala, mas ultrapassou o GD na instância grande.

Em relação ao tempo médio de execução, que não é limitado pelo orçamento de avaliações, o GA foi o mais lento nas três instâncias (1,25 s, 11,08 s e 161,76 s), chegando a cerca de 7,8 vezes o tempo do SA e 5,0 vezes o do GD na instância grande. O código do GA não utiliza a avaliação incremental e realiza operações de seleção e cruzamento sobre listas, mas não foi feita uma medição que isole a origem desse custo.

0.7 Efetividade do *Genetic Algorithm*

Nos experimentos realizados, o Algoritmo Genético com os parâmetros adotados ($N = 50$, $P_c = 0{,}8$, $P_m = 0{,}1$, torneio com $k = 3$, OX, *swap mutation* e elitismo de 2) não foi o melhor método em nenhuma instância. Na pequena e na média foi o pior dos três (p-valores abaixo

de 10^{-9} contra GD e SA), e na grande foi melhor que o GD, mas cerca de 2,3 vezes pior que o SA. Mesmo assim, na instância pequena o GA atingiu o valor de referência 1.875 em ao menos uma repetição (mínimo igual a 1.875), embora a mediana tenha sido 2.219.

A rápida queda de diversidade (de 1,0 para cerca de 0,12) e a proximidade entre a média da população e o melhor valor sugerem convergência prematura como explicação plausível para esse resultado, mas os experimentos não testaram essa hipótese diretamente. Também não foram variados os parâmetros do GA, e as opções PMX e seleção por roleta não foram incluídas no protocolo oficial, de modo que não é possível afirmar se outra configuração do GA teria desempenho melhor. Na instância grande, a curva de convergência mostra que o GA reduz o custo muito mais rapidamente que os demais no início (Figura 2), mas estagna, sendo superado pelo SA a partir de aproximadamente 170.000 avaliações.

Conclusões

Foram comparados o *Greedy Descent* com *Random Restart*, o *Simulated Annealing* e o Algoritmo Genético no problema de sequenciamento de manutenção, com três instâncias (12, 50 e 150 ativos) e orçamento de avaliações fixo, totalizando 420 execuções.

Na instância pequena, o GD e o SA encontraram o valor de referência 1.875 de forma praticamente equivalente ($p = 0{,}334$), e o GA ficou atrás. Nas instâncias média e grande, o SA obteve as menores médias, com diferença significativa em relação aos demais, e a vantagem sobre o GD aumentou com a escala (cerca de 1,19 vez na média e 11,0 vezes na grande). Na instância grande, o GD não chegou a nenhum ótimo local dentro do orçamento, e o GA, apesar de melhor que o GD, ficou cerca de 2,3 vezes acima do SA.

Do ponto de vista de exploração e intensificação, o SA fez uma transição gradual, com taxa de aceitação de piores caindo de cerca de 0,93 para cerca de 0,001, enquanto o GA perdeu diversidade rapidamente (de 1,0 para cerca de 0,12) e o GD dependeu quase exclusivamente da intensificação, com poucos reinícios nas instâncias maiores.

Como limitações, os p-valores não têm correção para múltiplas comparações, a instância grande teve apenas 10 repetições, não houve teste estatístico entre as variantes do GD, o valor 1.875 é a melhor solução encontrada e não um ótimo comprovado, e o GA foi avaliado com um único conjunto de parâmetros.

CRediT – Taxonomia de Contribuição de Autor

Caio César Araújo Guimarães: Conceituação, Visualização, Redação – rascunho original, Redação – revisão e edição.
Cecília Rodrigues Saraiva: Conceituação, Visualização, Redação – rascunho original, Redação – revisão e edição.
Stéphanie Pereira Barbosa: Conceituação, Software, Análise formal, Validação.
Thales Eduardo Dias de Souza: Conceituação, Análise formal, Validação, Redação – rascunho original, Redação – revisão e edição.

Declaração de Uso de IA Generativa

Durante a preparação deste trabalho, os autores utilizaram o Claude AI para corrigir erros de ortografia e formatação existentes no relatório.

A conversa completa com a IA está disponível neste link.

Referências

[1] Poole, D. L.; Mackworth, A. K. *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press, 2010.

[2] Eiben, A. E.; Smith, J. E. *Introduction to Evolutionary Computing*. Springer-Verlag, 2003.

[3] Gaspar-Cunha, A.; Takahashi, R. H. C.; Antunes, C. H. (Orgs.). *Manual de Computação Evolutiva e Metaheurística*. Editora UFMG, 2013.

[4] Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*. 3. ed. Pearson, 2016. Capítulo 4.

[5] Slides da disciplina Fundamentos de Inteligência Artificial, UFMG.